UNIVERSITY OF MACEDONIA

UNDERGRADUATE STUDIES PROGRAMME

OF THE DEPARTMENT OF APPLIED INFORMATICS

# A Multi-Tool Approach to Code Smell Detection and Metric Correlation

Degree Thesis of

Eleni Kagia

Thessaloniki, September 2024

## Abstract

Technical Dept estimation can be achieved by using special tools, that detect bad programming practices in code, called code smells. The tools, though, use different ways in detecting the smells making the process subjective and the results dependable on which detector is used. Studies, in order to fix this and get more certain results, combine multiple detectors. The present work adopts this idea and attempts to investigate the agreement between the four smell detectors Organic, PMD, CheckStyle and DuDe and the correlations of these tools with certain software metrics. In order to do that a tool was developed using the Java Programming Language that combines these detectors.

After an analysis of the topic and a reference to previous research, a description of the code and how it works is given. Then a case study is made on the results of the detection process and the conclusions that can be extracted from them.



## Key Words

Smell Detection, Code Smells, Software Metrics, Smell Detectors.

# Acknowledgement

I would like to express my sincere gratitude to my supervising Professor Ampatzoglou Apostolo for the trust he has shown me to carry out such an interesting topic.

I am also grateful to Nikolaidis Nikos for his continuous guidance, advice and effort to help me in the completion of the work.

Finally, I would like to thank my family for their selfless support throughout the years of my studies.

# Table of Contents

# List of Images

# List of Tables

# 1. Introduction

## 1.1. The problem - Importance of the issue

In software development the quality of the code plays a key role in the maintainability and ease of maintenance of systems. It is not unusual, though, for developers to sacrifice the quality for various reasons, creating hard-to-maintain code. This phenomenon is known as "Technical Debt". Technical Debt (TD) refers to the short-term benefits resulting from the neglect or postponement of best development practices, which in the long-term lead to increased maintenance costs and reduced software quality.

In order to estimate the TD, various static analysis tools were developed over time that look for violations of pre-defined rules, called code smell detectors. Despite the large number of these tools, the detection of code smells is a lengthy and complex process, and the results are not always certain. In order to fix this, many studies combine various smell detectors and check for the agreement between them, with their results to vary.

## 1.2. Purpose of the Thesis

In this work there will be an attempt to check the agreement between smell detection tools, Organic Smell Detector, PMD Smell Detector, CheckStyle Smell Detector and DuDe Smell Detector. In addition, a check will be made for correlations between the results of the detectors and various software metrics of the projects, that were analyzed.

## 1.3. Contribution

To carry out the work a tool was created, which combines the above-mentioned detectors. This tool detected smells in 25 different projects, written in the Java programming language, and exported the results in CSV files. After this process was completed, the agreement of the tools was calculated, along with the correlations with the software metrics.

## 1.4. Work structure

The paper is organized as shown below:

**Literature Review – Theoretical Background**: a comprehensive overview of the existing research on which the work is based and theoretical analysis of the design and construction of the code.
**Tools**: presentation of the tools used in the code.
**Tool Development**: analysis of the code architecture and presentation of the most important parts of it.
**Case Study**: detailed description of the extraction and processing of the data and the correlations found between them.
**Conclusion**: a brief review of the results and validation conditions of the paper.
**About Bibliography**: a list of the references used.

## 2. Literature Review - Theoretical Background

In code development, it is not enough for the code to be bug-free and run correctly. Its quality will determine whether it is maintainable and if it is, how easy it is to maintain. The development teams, though, often prioritize fast delivery over quality code, decision which later causes problems in maintaining or updating the code. Ward Cunningham [1] used the term Technical Dept (TD) to describe this situation and likened it to financial dept. According to the metaphor, one party borrows something, the development team borrows time, and repays it back with interest, in the future the team will repay it by putting more effort and time in maintaining the code.

It becomes quickly understandable thar TD must be avoided or at least minimized and to achieve that, poor quality code must be identified and refactored. Poor quality code can be identified by code smells. Code smells are indicators of potential issues in code that may not necessarily be bugs or errors but could lead to future problems in maintainability. They are like "red flags" that suggest something might be wrong with the code structure or design. At first an experienced developer could usually identify code smells simply by looking through the code. However, smell identification was automated, because code smells are frequent in even small-scale systems. The automation of smell detection happened with various tools, called Smell Detectors, that scan the code and find code smells in it.

There are many categories of code smells, each representing different aspects of poor coding practices. Table 1 shows all the kinds of smells that were used in this work and their definition.

| Code Smell | Description |
|---|---|
| Brain Class | A class that centralizes too much intelligence or business logic. |
| Brain Method | A method that is too complex and contains too much logic. |
| Class Data Should Be Private | When a class exposes its internal fields directly instead of using getter and setter methods. |
| Complex Class | A class that is overly complicated due to having too many responsibilities, attributes, or methods. |
| Data Class | A class that primarily contains fields and lacks significant methods to manipulate its data, often serving just as a data container. |
| Dispersed Coupling | A situation where a class is indirectly dependent on many other classes, often through global variables, static methods, or complex interactions. |
| Duplicate Code | Occurs when identical or very similar code appears in more than one place. |
| Feature Envy | Occurs when a method is more interested in the data and methods of another class than those of the class it is in, indicating that it might belong in the other class. |
| God Class | A class that has taken on too many responsibilities, often managing too many different parts of a program. |

| | |
|---|---|
| Intensive Coupling | A class that is heavily dependent on many other classes, making it highly susceptible to changes in those classes. |
| Lazy Class | A class that does very little, often only serving as a data holder without adding significant functionality, making it unnecessary or redundant. |
| Long Method | A method that is excessively long, doing too much in one place. |
| Long Parameter List | A method or function that takes too many parameters. It often indicates that some of the parameters could be grouped into a class or passed as an object. |
| Message Chain | Occurs when a client asks one object for another object, which asks yet another object for a different object, forming a chain of requests. |
| Refused Parent Bequest | A subclass that overrides or ignores the methods and properties it inherits from its parent class, indicating that the subclass does not really fit the parent class's abstraction. |
| Shotgun Surgery | Occurs when a single change in one part of the code requires making lots of small changes in multiple other classes. |
| Spaghetti Code | Code that is difficult to follow due to a lack of structure, often resulting from poor design decisions, lack of adherence to coding standards, or ad-hoc fixes. |
| Speculative Generality | Occurs when code is written to handle situations that may never occur or includes abstract classes, interfaces, or methods that are not currently needed but might be useful in the future, leading to unnecessary complexity. |

*Table 1 – Code Smells detected in this work and their description.*

While the Smell Detectors are helpful in identifying bad code, due to the differences between them, one must be extra careful in choosing which one to use as the adopted detection techniques determine the obtained results. Even though, the whole process of choosing the detector, analyze the code and extract the conclusions from the results is time-consuming and there is no guarantee that the detector found all problematic pieces of code.

For this reason, many studies have adopted the practice of creating a tool that uses more than one Smell Detector. The idea behind this, is that the use of more than one Smell Detector will lead to more reliable results. The results of these studies vary, due to the Smell Detectors they use. The detectors not only identify different kinds of smells, they also use different practices to do it. So, it is only natural that there is larger agreement between a certain group of detectors than in another. For example, Fontana et al. [2] in their work used four tools, JDeodorant, inFusion, PMD and CheckStyle, that identified six code smells in total. Their study showed even though there is a high detection agreement, inconsistencies exist across different systems and versions: "*The values range from 85% to 100%, giving an idea of strong agreement among the tested tools. These values indeed hide the fact that the data are unbalanced. The large majority of the agreement among the tools is on false values: Most of the classes (or*

*methods) are classified as not containing any smell.*". Rasool and Arshad [3] reviewed code smell detection methods and found significant variation in the results of different tools. They confirmed this by testing three tools—CheckStyle, JDeodorant and PMD—on four common code smells, observing inconsistencies among the results. One more study by Hamid et al. [4], compared two tools, JDeodorant and inCode and after an analysis on two common code smells, Feature Envy and God Class, the results differed. Fernades et al. [5] in their study specifically evaluated four prominent tools: inFusion, JDeodorant, PMD, and JSpIRIT, focusing on their capabilities in detecting Large Class and Long Method smells. The findings indicated a high level of agreement among the tools in detecting these smells, with JDeodorant identifying the most instances, while PMD demonstrated superior precision. Lastly, Ichtsis et al. [6] combined six detector tools CheckStyle, DuDe, PMD, JDeodorant, JSpIRIT and Organic. The results showed varying levels of agreement between different tools in detecting code smells, with some tools identifying significantly more instances of certain smells than others. Of course, this study suffered from threats to external validity as only one project was analyzed, making the results unreliable. Most of the studies attributed the different levels of agreement in the different smell detection methods. For example, some detectors use metrics to identify the "God class" smell while others compute it as a class that can be decomposed into other classes. Even the different thresholds for the metrics affect the results.

| **Paper** | **Tools** | **Results (as described by the authors)** |
|---|---|---|
| Fontana et al. [2] | JDeodorant, inFusion, PMD, CheckStyle | High detection agreement, but with inconsistencies across different systems and versions |
| Rasool and Arshad [3] | CheckStyle, JDeodorant, PMD | Inconsistencies among the results. |
| Hamid et al. [4] | JDeodorant, inCode | The results differed due to different techniques of the two detectors. |
| Fernades et al. [5] | inFusion, JDeodorant, PMD, JSpIRIT | High level of agreement among the tools. |
| Ichtsis et al. [6] | CheckStyle, DuDe, PMD, JDeodorant, JSpIRIT, Organic | Varying levels of agreement between different tools. |

*Table 2 – Summary of related papers and their results.*

The present work may be following on the steps of these studies, in comparing various smell detector tools, but it also tries to find correlations between the tools and various software metrics. Software metrics are quantitative measures used to assess various aspects of software development and performance. These metrics help in evaluating the quality, complexity, efficiency, and maintainability of software systems. They provide objective data that can guide decision-making in software development, help identify potential issues, and facilitate process improvement. The goal in using the metrics is to improve the reliability of the detection process and to predict the areas of code that are more likely to contain quality problems.

The metrics used in the present work, were taken from the study by Tsoukalas et al. [7] and they range from refactoring operations to process metrics and from code issues to source code metrics. Tsoukalas et al. exported these metrics from the same 25 projects the present work analyzes, in order to create a Machine Learning model for TD identification. The table below shows which metrics were used, along with a description of each one of them.

| Metrics | Description |
|---|---|
| commits_count | Total number of commits made to a file in the evolution period. |
| code_churn_avg | Average size of a code churn of a file in the evolution period. |
| contributors_count | Total number of contributors who modified a file in the evolution period. |
| contributors_experience | Percentage of the lines authored by the highest contributor of a file in the evolution period. |
| hunks_count | Median number of hunks made to a file in the evolution period. A hunk is a continuous block of changes in a diff. This number assesses how fragmented the commit file is (i.e., lots of changes all over the file versus one big change). |
| issue_tracker_issues | Total number of times a file name has been reported in the project's Jira or GitHub issue tracker (mentioned within either the title or the body of the registered issue) in the evolution period. |
| cbo | Coupling Between Objects. This metric counts the number of dependencies a file has. |
| wmc | Weight Method Class or McCabe' s complexity. This metric counts the number of branch instructions in a file. |
| dit | Depth Inheritance Tree. This metric counts the number of "fathers" a file has. All classes have DIT at least 1. |
| rfc | Response for a Class. This metric counts the number of unique method invocations in a file. |
| lcom | Lack of Cohesion in Methods. This metric counts the sets of methods in a file that are not related through the sharing of some of the file's fields. |
| max_nested_blocks | Highest number of code blocks nested together in a file. |
| total_methods | Total number of methods in a file. |
| total_variables | Total number of declared variables in a file. |
| total_refactorings | Total number of refactorings for a file in the evolution period. |
| duplicated_lines_density | Percentage of lines in a file involved in duplications (100 * duplicated lines / lines of code). The minimum token length which should be reported as a duplicate is set to 100. |

| | |
|---|---|
| comment_lines_density | Percentage of lines in a file containing either comment or commented-out code (100 * comment lines / total number of physical lines). |
| ncloc | Total number of lines of code in a file, ignoring empty lines and comments. |
| Max_Ruler | The Max-Ruler class profile, refers to modules whose reference assessment type indicates a high amount of TD based on the results of the tree applied tools. |

*Table 3 - Metrics used and their description.*

This thesis highlights the crucial role that smell detectors and the identification of code smells have in the management of TD. Although code smells can be used as markers for possible maintenance problems, the effectiveness of smell detectors varies greatly depending on the methodology used. This paper's latter sections will compare and contrast four smell detectors and analyze their agreement. The correlations between code smells and software metrics will also be investigated in order to offer greater insights into efficiently controlling TD and enhancing overall code quality.

# 3. Tools

The Java project developed in the context of the thesis is based on the Eclipse plugin of the paper of Ichtsis et al. and like the plugin it combines various smell detector tools. The tools used in the code are Organic Smell Detector, PMD Smell Detector, CheckStyle Smell Detector and DuDe Smell Detector. However, there are two tools, JDeodorant Smell Detector and JSpIRIT Smell Detector, that the plugin uses that are not included in the project for reasons explained in Section 6. Below is a description of each of these tools and what kind of smells they detect in the project. A table is also given with the types of smells that are in common between the detectors. It is important to be known that all the detectors work with their default settings and nothing was changed in their rules.

## 3.1. Organic Smell Detector

Organic aims to systematically collect and analyze code smells from Java projects using a suite of command-line tools, emphasizing efficiency and automation [9]. This toolchain operates by parsing source code and applying a series of predefined rules and heuristics to identify various types of code smells. For example, Organic in order to find a “Class Data Should Be Private” type of smell, it checks if a class has at least one public field. For a “Complex Class” type of smell, it checks if a class has at least one method for which McCabe cyclomatic complexity, a software metric used to indicate the complexity of a program, is higher than 10. “Feature Envy” smell is detected if all methods have more calls with another class than the one they are implemented, while the “Lazy Class” smell, if all classes have LOCs lower than the first quartile of the distribution of LOCs for all systems classes.

The rule for “Long Method” smell is for methods to have LOCs higher than the average of the system and for “Long Parameter List” is for methods to have a number of parameters higher than the average of the system. “Message Chain” smell is detected when the number of called methods in a chain is larger than three and “Refused Parent Bequest” when all classes override more than half of the methods inherited by a superclass. For a “Spaghetti Code” smell, Organic checks if a class implements at least two long methods interacting between them through method calls or shared fields, and for a “Speculative Generality” if a class declared as abstract has less than three children classes using its methods. The rule for “Brain Method” smell is for a method to be overloaded with software features, for “Dispersed Coupling” smell a method calls too many methods and for “Intensive Coupling” a method depends much on other ones. Lastly “Brain Class” smell is detected when a class is overloaded with software features, “Data Class” smell when a class contains only data management features and “God Class” when a class has too many software features.

## 3.2. PMD Smell Detector

PMD (Programming Mistake Detector) is an extensible multilanguage static code analyzer. It finds common programming flaws like unused variables, empty catch blocks, unnecessary object creation, and so forth. [8]. It can support smell detection on many

languages with different set of rules for each one of them. The following table shows the categories into which the rules are divided along with the priority of each one.

| Category | Description | Priority |
|---|---|---|
| Best Practices | Rules which enforce generally accepted best practices. | Medium (3) |
| Code Style | Rules which enforce a specific coding style. | Medium (3) |
| Design | Rules that help the discovery of design issues. | High (1) |
| Documentation | Rules that are related to code documentation. | Medium (3) |
| Error Prone | Rules to detect constructs that are either broken, extremely confusing or prone to runtime errors. | Medium (3) |
| Multithreading | Rules that flag issues when dealing with multiple threads of execution. | Medium (3) |
| Performance | Rules that flag suboptimal code. | Medium (3) |
| Security | Rules that flag potential security flaws. | Medium (3) |

*Table 4 – Categories of PMD tool rules.*

Within the scope of code smell detection in this work, PMD focuses on specific types: Duplicate Code, God Class, Long Method, and Long Parameter List.

PMD finds “Duplicate Code” by using a special tool called Copy/Paste Detector (CPD). CPD analyzes source code files and identifies pieces of code that are syntactically identical or nearly identical. For the “God Class” smell it uses the following metrics:

- WMC: a class complexity measure
- ATFD: a measure of how much external data the class uses
- TCC: a measure of how tightly related the methods are

The rule belongs in the “Design” category and for identifying a God class it looks for classes which have at the same time high WMC, high ATFD and low TCC.

For the detection of the “Long Method” smell, PMD uses the “ExcessiveMethodLength” rule, which falls into the “Design” category. This rule works by counting the number of lines in each method, including statements, blocks, loops and comments within the method. If the line count exceeds a certain limit PMD flags the method. The default limit is typically 80 lines. Similar is the rule for detecting the “Long Parameter List” smell, the “ExcessiveParameterList”. It belongs, also, in the “Design” category and works by checking the number of formal parameters declared in a method or constructor. If the count exceeds the limit, usually set around 5 to 7 parameters, the detector flags the method.

### 3.3. CheckStyle Smell Detector

CheckStyle is a static code analysis tool used in software development for checking if Java source code is compliant with specified coding rules. The checks performed by CheckStyle are mainly limited to the presentation of the code. These checks do not

confirm the correctness or completeness of the code [10]. The checks are categorized by functionality. The following table provides the categories along with a description for them.

| Category | Description |
|---|---|
| Annotations | Ensures proper use and placement of annotations. |
| Block Checks | Focuses on the structure and layout of code blocks, such as braces and block formation. |
| Class Design | Enforces design principles related to class structure, including inheritance, visibility, and cohesion. |
| Coding | Covers general coding practices, including avoiding magic numbers and proper exception handling. |
| Headers | Ensures files contain appropriate header comments, often used for license or file information. |
| Imports | Enforces rules regarding the usage and ordering of import statements to maintain clean dependencies. |
| Javadoc Comments | Ensures that code is properly documented with Javadoc for clarity and maintainability. |
| Metrics | Focuses on measuring code metrics, such as cyclomatic complexity, to identify potential issues in the code. |
| Miscellaneous | Includes various other checks that contribute to overall code quality but don't fit into other categories. |
| Modifiers | Enforces correct usage of access modifiers (e.g., public, private) to ensure proper encapsulation. |
| Naming Conventions | Enforces consistent naming patterns for classes, methods, variables, etc., to improve readability. |
| Regexp | Allows for custom checks based on regular expressions to enforce specific code patterns. |
| Size Violations | Focuses on the size of code elements (e.g., files, methods) to prevent overly large and complex components. |
| Whitespace | Ensures proper usage of whitespace, contributing to consistent code formatting and readability. |

*Table 5 – Categories of checks performed by the CheckStyle tool.*

In the project it detects God Class, Long Method and Long Parameter List types of smells. “God Class” smell is detected using the “FileLength” module, which is responsible for enforcing restrictions on the length of code files. A module refers to a specific component or plugin that performs a particular check or validation against the source code. This module belongs in the “Size Violations” category and its goal, here, is to ensure that source code files do not become too large. In order for CheckStyle to detect a “Long Method” type of smell, it uses the “MethodLength” module. This module belongs, also, in the “Size Violations” category and is responsible for checking the length of the methods. It uses the Abstract Syntax Tree (AST) representation of the code to count the number of lines in each method. For the “Long Parameter List” smell the detector uses the “ParameterNumber” module, which belongs in the “Size Violations” category. It checks the number of parameters that a method receives. Just like

the previous module, it uses the AST representation of the code to count the parameters on a method.

### 3.4. DuDe Smell Detector

DuDe (Duplication Detector), according to its creators, is an automated line-based language-independent code duplication detector designed to identify and analyze code duplications [11]. As its primary function, DuDe focuses exclusively on identifying Duplicate Code. It uses configurable parameters to customize detection results, allowing users to filter out the least important copies and recover copy chains. The tool provides a list of duplication chains instead of a graphical representation, allowing engineers to more efficiently analyze candidate chains.

DuDe has three phases of detection process. The first is “Code Preprocessing”, in which the tool reads line by line the source-files and eliminates the white spaces so that the various indentation styles would not make any difference. Next it eliminates the noise, the lines of code made of syntactic elements like a single closing brace or empty lines. The second phase is the “Populate the scatter-plot” phase. The tool uses a scatter-plot approach, it compares every line of relevant code with every line of code in the project and stores the result of the comparison in a Matrix cell M[i,j] (comparison between the relevant line i and the relevant line j). Then the matrix is divided in two symmetrical areas around the main diagonal and works with one of them in order to avoid storing redundant information. The third and final phase is the “Build the duplication chains” phase. Here the tool starts from the upper left corner of the matrix and looks for the first marked cell. If this chunk exceeds the minimum SEC, Size of Exact Chunk, threshold (= 3), it will be marked as a valid exact chunk and will be the starting point for a duplication chain, which will be further extended up to the first unmarked cell on the main diagonal direction. Then it searches “nearby” for another valid chunk to add to the chain, if it does not exceed the LB, Line Bias, threshold (= 2). The search continues like this until the expansion of the chain candidate is not possible anymore. In the end the tool measures the size of the potential duplication chain and if it is longer than the minimum SDC, Size of Duplication Chain, threshold (= 8) it is declared as a valid duplication chain.

This tool has the advantage that it can detect smaller duplication fragments, otherwise hardly noticeable, which belong to a bigger, more important duplication block. This way it provides a bigger picture to the developer, helping him make more informed refactoring decisions. The downside, though, is that because of its rather high granularity of comparison, the tool cannot detect renamed variables. Only some of them are covered by duplication chains of the special type modified.

### 3.5. JDeodorant Smell Detector

JDeodorant is an Eclipse plug-in that identifies design problems in software, known as bad smells, and resolves them by applying appropriate refactorings. It employs a variety of novel methods and techniques in order to identify code smells and suggest the appropriate refactorings that resolve them. For the moment, the tool identifies five

kinds of bad smells, namely Feature Envy, Type Checking, Long Method, God Class and Duplicated Code [12].

Feature Envy problems are resolved by appropriate Move Method refactorings. Type Checking problems are resolved by appropriate Replace Conditional with Polymorphism and Replace Type code with State/Strategy refactorings. Long Method problems are resolved by appropriate Extract Method refactorings. God Class problems are resolved by appropriate Extract Class refactorings and Duplicated Code problems are resolved by appropriate Extract Clone refactorings.

JDeodorant encompasses a number of innovative features:

- Transformation of expert knowledge to fully automated processes.
- Pre-Evaluation of the effect for each suggested solution.
- User guidance in comprehending the design problems.
- User friendliness (one-click approach in improving design quality).

### 3.6. JSpIRIT Smell Detector

JSpIRIT (Java Smart Identification of Refactoring opportunITies) is a tool that it detects ten types of smells: Brain Class, Brain Method, Data Class, Disperse Coupling, Feature Envy, God Class, Intensive Coupling, Refused Parent Bequest, Shotgun Surgery and Tradition Breaker. One of the unique aspects of this tool is its ability to rank or prioritize the detected code smells. To prioritize the code smells JSpIRIT provides several kinds of rankings that use different criteria such as the history of the application, the relevance of the kind of code smell, or modifiability scenarios.

Another unique characteristic is its ability to identify agglomerations of code smells. Agglomerations are groups of inter-related code smells (e.g., syntactically-related code smells within a component) that likely indicate together the presence of an architectural problem. JSpIRIT supports 2 kinds of agglomerations: smells within a component and smells in a hierarchy. When code smells are implemented by the same architectural component, they belong in the “smells within a component” category. Specifically, the tool looks for code smells that are syntactically related or for code elements infected by the same type of code smell. The category "smells in a hierarchy" includes smells that occur across the same inheritance tree involving one or more components. The rationale is that a recurring introduction of the same smell in different code elements might represent a bigger problem in the hierarchy [13].

| Detector Names | God Class | Long Method | Long Parameter List | Duplicate Code |
|---|---|---|---|---|
| Organic | ✓ | ✓ | ✓ | × |
| PMD | ✓ | ✓ | ✓ | ✓ |
| CheckStyle | ✓ | ✓ | ✓ | × |
| DuDe | × | × | × | ✓ |

*Table 6 - Kinds of smells that the detectors have in common.*

Generally, the smell detection tools used have different approaches and complementary capabilities, covering a wide range of code problems. Organic uses metrics and heuristic rules for detecting complex design smells such as God class and Feature Envy. PMD and CheckStyle, on the other hand, focus on more common programming errors and detect through metrics and static rules. Finally, DuDe specializes in the "Duplicate Code" type of smell using an algorithmic approach based on comparing lines of code. Despite their differences, the combination of these detectors can provide a more comprehensive and balanced coverage of code smells, that could not be achieved just by using one of them.

Next sections describe how the developed tool combines the smell detectors and discuss the level of agreement between them.

# 4. Development of the Tool

Full code can be found at: KagiaEleni/SmellDetector: A Java Project that combines existing code smell detector tools. (github.com).

This section analyses the code developed for the thesis and the architectural decisions made for it. Some important pieces of code are also given to make it easier to understand.

## 4.1. Architectural Decisions

As stated in previous section, the java project created here, is based on the Eclipse plugin of the paper of Ichtsis et al. As stated in the paper, only one project was analyzed and this is because as an Eclipse plugin it can only be used on java projects imported into Eclipse, which limited the number of projects that can be analyzed. What was more problematic, though, it was that it could not handle big and complicated projects. Therefore, one of the most crucial architectural decisions was to change the plugin into a simple java project. This allowed the analysis of the 25 projects that did not need to be imported in the Eclipse.

In the change to a java project, it was quickly noticed that the variable used to import the program that was to be parsed, could not be used in the project. This is because the variable, IJavaProject, is a part of Eclipse Java Development Tools (JDT) API, but not in the standard Java Development Kit (JDK). Based on this information, it was decided to replace this variable with a new variable of type String, that would contain the project directory of the program to be analyzed. The “java.nio.file” package was also used to handle files and directories. Another type that could not be used was the Bundle type. The Bundle variable is used to manage and access resources within the plugin or the environment where the code is executed. In an Eclipse-based environment, a Bundle represents a unit of modularization (typically a plugin). This Bundle object is used to locate and load files or resources that are bundled with the plugin, such as configuration files, libraries, or scripts necessary for the smell detection process. This variable did not need to be replaced, as the files that it helped load in the code were inside the folder of the java project and therefore easily accessible. The next section analyses these changes in more detail and provides pieces of code that have been changed, for better understanding.

Another decision, was to add a simple graphical user interface to make it easier to use. The user is able to insert the project directory of the program he wishes to analyze in a Text field in the GUI, a directory in a second Text field where the files with the results will be saved and then choose from a list with which detector it will be analyzed. After the detection phase is completed, a table with the results is shown along with some filters for better data analysis. This was chosen so that the user does not have to change the code directly, and because it is a more user-friendly approach to the user than communicating via the console.

## 4.2. Tool Development

The Java project consists of twelve classes: Main, SmellResultsFrame, SmellDetectionManager, SmellDetector, SmellType, Smell, OrganicSmellDetector, PMDSmellDetector, CheckStyleSmellDetector, DuDeSmellDetector, Utils and SmellExporter.

The class “Main” is responsible for creating the GUI application. As mentioned, the application is designed to manage the detection of code smells. The main window of the application, named “Smell Detector Application” is the main window and it organizes all components and panels. The class creates three different panels. The first panel is the Project Directory panel and contains elements for defining the project directory. A JTextField for the user to write the project directory, a “Browse” JButton if the user wants to browse his\her computer’s file system for the project directory and a “Change” JButton, that sets the project directory the user chose. The second panel contains the same elements (JTextField, two JButtons), but for setting the export directory, where the files with the results from the detectors will be saved. The third and last panel is the Options panel. This panel contains two JButtons, one that leads the user into a new window to choose a Smell Detector (“Select Smell Detector”) and one that terminates the application (“Exit”). The “Select Smell Detector” first checks if the project directory and the export directory have been set and if they have, it creates the new window “Select Smell Detector”, that has two panels. The first one contains a JRadioButton element where the user can choose to run one of the four detectors or all of them together. The second panel has two JButtons, one that shuts the window (“Back”) and one that starts the smell detecting process (“Select”). When the “Select” Button is pressed the class creates a SmellDetectionManager object and from there the smell detection begins. Once it’s finished the class creates a SmellResultsFrame object and a SmellExporter object to display and save the results.

The “Smell” class represents a code smell detected in a project and saves details about it, such as the type of smell, the affected code elements, and the tools that detected it. The class also supports code smell instances related to duplicated code, with additional fields for duplication group information. The “Smell” objects are created using the “Builder” pattern. “Builder” class provides a flexible way to create “Smell” objects with varying properties, particularly useful when only some fields need to be set. It is equipped with various methods that set the necessary fields of the smell: DuplicationGroupId, ClassName, MethodName, TargetFile, StartLine and EndLine. The method “build” creates and returns a new Smell instance using the provided values. As for the “Smell” class it is equipped with methods that get or add a certain field. The method “equals” makes sure that smells with the same attributes are not duplicated, and the method “hashCode” makes sure that the smells are stored in an efficient way.

The “SmellType” enum represents various types of code smells that can be detected in the project. The method “getName” returns the human-readable name of the smell type, while the “getSmellTypeFromChoice” takes an integer smellChoice as input and returns the corresponding SmellType based on the order in which they are declared in the enum. It uses the ordinal position of each enum constant, adjusting for zero-based

indexing by adding 1 to match typical user input expectations (e.g., user chooses 1 for the first smell type). If the choice does not match any existing enum, the method returns null.

The “SmellDetector” class is an abstract base class that defines the common interface and behavior for all specific smell detectors used in the project. Each concrete subclass of SmellDetector (PMDSmellDetector, CheckStyleSmellDetector, DuDeSmellDetector, OrganicSmellDetector) must implement these methods to perform their specific code smell detection tasks. There is the “getSupportedSmellTypes” method, that returns a Set of “SmellType” that the subclass can identify. The “getDetectorName” method returns the name of the smell detector and the “getDetectedSmells” method returns the smells that have been detected by him. Key Component is the method “findSmells”. This abstract method is responsible for detecting code smells of a given type in the project. Implementing classes define the logic for how the detector identifies these smells.

```java
public abstract class SmellDetector {

    /**
     * A method that returns all the code smell types that can be found from the
     * detector.
     *
     * @return a {@code Set} of smell types
     */
    public abstract Set<SmellType> getSupportedSmellTypes();

    /**
     * A method that returns the name of the detector, which is used to differentiate
     * the different detectors.
     *
     * @return the name of the detector
     */
    public abstract String getDetectorName();

    /**
     * A method that is responsible for finding the code smells based on the
     * given smell type.
     *
     * @param smellType the smell type to check for
     * @param detectedSmells a {@code Map} from smellType to a {@code Set} of detected smells
     */
    public abstract void findSmells(SmellType smellType, Map<SmellType, Set<Smell>> detectedSmells) throws Exception;

    public abstract Map<SmellType, Set<Smell>> getDetectedSmells();

}
```

*Image 1 – The SmellDetector Class.*

The “SmellDetectionManager” is responsible for managing the detection of code smells in a project. Its constructor calls for the method “initialiseNecessaryClassFields”, which is responsible for initializing the correct detector/s depending on the user's choice. For each detector the class first creates an instance of the class of the detector and calls for the method “findSmells”. After the end of the detection, the “updateDetectedSmells” is used to update the map with the smells (image 2). The smells are stored in a Map<SmellType, Set<Smell>> called detectedSmells. This map is a class-level variable of the “SmellDetectionManager” class, and it associates each “SmellType” with a Set of Smell objects.

```
if (useAllDetectors || smellTypeToBeDetected == SmellType.ORGANIC) {
    OrganicSmellDetector organicDetector = new OrganicSmellDetector(projectDirectory);
    try {
        organicDetector.findSmells(smellTypeToBeDetected, detectedSmells);
        updateDetectedSmells(organicDetector);
    } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    smellDetectors.add(organicDetector);
}
```

*Image 2 – Example of how the "SmellDetectorManager" creates and runs the detectors (here Organic).*

```
private void updateDetectedSmells(SmellDetector detector) {
    if (detector != null) {
        Map<SmellType, Set<Smell>> detectedSmells = detector.getDetectedSmells();

        // Check if the map is empty
        if (detectedSmells == null || detectedSmells.isEmpty()) {
            return; // Exit early if there are no smells to process
        }

        for (Map.Entry<SmellType, Set<Smell>> entry : detectedSmells.entrySet()) {
            SmellType smellType = entry.getKey();
            Set<Smell> smells = entry.getValue();
            detectedSmells.merge(smellType, smells, (existingSmells, newSmells) -> {
                existingSmells.addAll(newSmells);
                return existingSmells;
            });
        }
    }
}
```

*Image 3 – The "updateDetectedSmells" method of the "SmellDetectorManager" class.*

The classes "OrganicSmellDetector," "PMDSmellDetector," "CheckStyleSmellDetector," and "DuDeSmellDetector," subclasses of the class "SmellDetector," are responsible for managing the respective smell detectors. Each class defines its own logic on the methods that it inherits and has extra when needed. All detectors initialize with the String projectDirectory and a HashMap where the detected smells are stored.

For example, the "findSmells" method in the "OrganicSmellDetector" class is responsible for detecting code smells in a Java project using the Organic tool. It begins by loading all the types (classes) in the specified project directory, then collects metrics for these types and detects any associated code smells. Once the smells are detected, the method filters the results to include only the smelly classes. For each smelly class, it determines the file path and uses the "extractSmells" method to process the detected smells. This method, "extractSmells", takes the detected smells from the Organic tool and maps them to the system's SmellType using a predefined mapping. It checks if the detected smell matches the type of smell currently being searched for (or if all smells are being detected). It then determines if the smell is associated with the entire class or just a method within the class. Depending on this, it creates a Smell object, associating it with the appropriate class or method, and stores it in the detectedSmells map. This process ensures that all relevant smells are accurately detected

and recorded, ready for further analysis or reporting. Images 4 and 5 and show the methods “findSmells” and “extractMethods”.

```
public void findSmells(SmellType smellType, Map<SmellType, Set<Smell>> detectedSmells) throws Exception {

    List<String> sourcePath = new ArrayList<>();
    sourcePath.add(projectDirectory);
    Organic organicPlugin = new Organic();

    List<Type> classTypeDeclarations = organicPlugin.loadAllTypes(sourcePath);
    organicPlugin.collectTypeMetrics(classTypeDeclarations);
    organicPlugin.detectSmells(classTypeDeclarations);
    classTypeDeclarations = organicPlugin.onlySmelly(classTypeDeclarations);

    for (Type classTypeDeclaration : classTypeDeclarations) {
        String fullClassPath = classTypeDeclaration.getFullyQualifiedName();
        String className = fullClassPath.substring(fullClassPath.lastIndexOf('.') + 1);

        File sourceFile = classTypeDeclaration.getSourceFile().getFile();

        // Adjust the path accordingly, assuming getFilePath() returns the path as a String
        String FilePath = sourceFile.getAbsolutePath();

        // Convert to File object
        File targetFile = new File(FilePath);

        // Extract smells using the file path
        extractSmells(smellType, detectedSmells, classTypeDeclaration.getSmells(), className, targetFile);

        for (Method methodTypeDeclaration : classTypeDeclaration.getMethods()) {
            extractSmells(smellType, detectedSmells, methodTypeDeclaration.getSmells(), className, targetFile);
        }
    }
}
```

*Image 4 – The “findSmells” method of the “OrganicSmellDetector” class.*

```
private void extractSmells(SmellType smellType, Map<SmellType, Set<Smell>> detectedSmells,
          List<br.pucrio.opus.smells.collector.Smell> toolSmells,String className, File targetFile) throws Exception {
    for(br.pucrio.opus.smells.collector.Smell smell: toolSmells) {
        SmellType detectedSmellType = MAP_FROM_SMELLNAME_TO_SMELLTYPE.get(smell.getName());

        int startingLine = smell.getStartingLine();

        if(Utils.isClassSmell(detectedSmellType)) {
            Utils.addSmell(detectedSmellType, detectedSmells, getDetectorName(),
                    Utils.createSmellObject(detectedSmellType, className, targetFile, startingLine));
        } else {
            String methodName = (String) Utils.extractMethodNameAndCorrectLineFromFile(targetFile, startingLine)[0];
            Utils.addSmell(detectedSmellType, detectedSmells, getDetectorName(),
                    Utils.createSmellObject(detectedSmellType, className, methodName, targetFile, startingLine));
        }
    }
}
```

*Image 5 – The “extractSmells” method of the “OrganicSmellDetector” class.*

The “PMDSmellDetector” class uses the methods very differently. As shown in image 6, when the “findSmells” method is called, it triggers the detection of the smells using the “detectPMDSmells” method. As for the duplicate kind of smells it uses a different method, the “detectCPDDuplicates”. The “detectPMDSmells” method, as shown in image 7, first loads necessary library files and performs the detection by executing a command-line tool.

```
public void findSmells(SmellType smellType, Map<SmellType, Set<Smell>> detectedSmells) throws Exception {

    detectPMDSmells(smellType, detectedSmells);
    detectCPDDuplicates(detectedSmells);

    this.detectedSmells = detectedSmells;
    System.out.println("End PMD");

}
```

*Image 6 – The “findSmells” method of the “PMDSmellDetector” class.*

The output is saved in an XML document which is parsed by the “extractSmells” method, in order to extract the smells. This method works in a completely different way from the one in Organic. It processes An XML document to detect and categorize the smells, by retrieving <file> nodes from the document, extracting information such as the file name and path. Then, for each file, it looks for <violation> nodes that contain details about the code smells, such as the start line and the name of the class or method, it creates smell objects from them and adds them to the detectedSmells map, grouping the smells by type (SmellType).

```
private void detectPMDSmells(SmellType smellType, Map<SmellType, Set<Smell>> detectedSmells) throws Exception {
    File pmdBatFile = new File(getClass().getClassLoader().getResource("pmd-bin-6.37.0/bin/pmd.bat").toURI());
    File pmdConfigFile = new File(getClass().getClassLoader().getResource("pmd-bin-6.37.0/bin/pmd-config.xml").toURI());
    File pmdCacheFile = new File(getClass().getClassLoader().getResource("pmd-bin-6.37.0/bin/pmd-cache.txt").toURI());

    String pmdOutput = Utils.runCommand(buildMainToolCommand(pmdBatFile, pmdConfigFile, pmdCacheFile), null, true);
    Document pmdXmlDoc = Utils.getXmlDocument(pmdOutput);

    extractSmells(smellType, pmdXmlDoc, detectedSmells);
}
```

*Image 7 – The “detectPMDSmells” method of the “PMDSmellDetector” class.*

```
private void extractSmells(SmellType smellType, Document xmlDoc, Map<SmellType, Set<Smell>> detectedSmells) throws Exception {
    NodeList fileNodes = xmlDoc.getDocumentElement().getElementsByTagName("file");
    for (int fileIndex = 0; fileIndex < fileNodes.getLength(); fileIndex++) {
        Node fileNode = fileNodes.item(fileIndex);
        String fileName = fileNode.getAttributes().getNamedItem("name").getNodeValue();
        String filePath = fileName.substring(fileName.indexOf("\\", fileName.indexOf("\\", projectDirectory.length())));
        File targetFile = new File(filePath); // Create a java.io.File object here

        NodeList violationNodes = fileNode.getChildNodes();
        for (int i = 0; i < violationNodes.getLength(); i++) {
            Node violationNode = violationNodes.item(i);
            // This is needed because there are unexpected children, e.g. #text
            if (!violationNode.getNodeName().equals("violation"))
                continue;

            SmellType detectedSmellType = MAP_FROM_DECTECTED_SMELLS_TO_SMELLTYPE.get(violationNode.getAttributes().getNamedItem("rule").getNodeValue());
            if (smellType != SmellType.ALL_SMELLS && smellType != detectedSmellType)
                continue;

            int startLine = Integer.parseInt(violationNode.getAttributes().getNamedItem("beginline").getNodeValue());
            String className = violationNode.getAttributes().getNamedItem("class").getNodeValue();

            if (detectedSmellType == SmellType.GOD_CLASS) {
                Utils.addSmell(detectedSmellType, detectedSmells, getDetectorName(),
                        Utils.createSmellObject(SmellType.GOD_CLASS, className, targetFile, startLine));
            } else {
                String methodName = violationNode.getAttributes().getNamedItem("method").getNodeValue();
                Utils.addSmell(detectedSmellType, detectedSmells, getDetectorName(),
                        Utils.createSmellObject(detectedSmellType, className, methodName, targetFile, startLine));
            }
        }
    }
}
```

*Image 8 – The “extractSmells” method of the “PMDSmellDetector” class.*

As for the “detectCPDDuplicates” method, it works the same. It loads the necessary library files and by executing a command-line tool, it performs the detection of duplicate code. The results are again saved in an XML document, but for their extraction a different method is used, the “extractDuplicates” method. The method works by firstly determining the largest duplicate group ID using a helper method. Then, it retrieves the <duplication> nodes from the XML and for each duplicate code, parses the <file> elements to extract information such as the start and end line of the duplicate code. For each duplicate instance, it creates a scent object and adds it to the detectedSmells map, grouping the duplicates by their group ID and updating the ID for the next group.

```
private void extractDuplicates(Document xmlDoc, Map<SmellType, Set<Smell>> detectedSmells,
        String projectDirectory) throws Exception {
    int duplicationGroupId = Utils.getGreatestDuplicationGroupId(detectedSmells);

    NodeList duplicationNodes = xmlDoc.getDocumentElement().getElementsByTagName("duplication");
    for (int i = 0; i < duplicationNodes.getLength(); i++) {

        NodeList fileEntries = duplicationNodes.item(i).getChildNodes();
        for (int j = 0; j < fileEntries.getLength(); j++) {
            Node fileEntry = fileEntries.item(j);
            // Each duplication has a <codefragment> element which only contains the duplicate code
            // and doesn't have any attributes. So this is recognized this way and is ignored
            if (!fileEntry.hasAttributes())
                continue;

            NamedNodeMap fileEntryAttributes = fileEntry.getAttributes();
            int startLine = Integer.parseInt(fileEntryAttributes.getNamedItem("line").getNodeValue());
            int endLine = Integer.parseInt(fileEntryAttributes.getNamedItem("endline").getNodeValue());
            String path = fileEntryAttributes.getNamedItem("path").getNodeValue();
            String className = path.substring(path.lastIndexOf("\\") + 1);
            String filePath = path.substring(path.indexOf("\\", path.indexOf(projectDirectory)));
            File targetFile = new File(filePath);

            Utils.addSmell(SmellType.DUPLICATE_CODE, detectedSmells, getDetectorName(),
                    Utils.createSmellObject(SmellType.DUPLICATE_CODE, duplicationGroupId, className,
                            targetFile, startLine, endLine));
        }

        duplicationGroupId++;
    }
}
```

*Image 9 – The "extractDuplicates" method of the "PMDSmellDetector" class.*

The "CheckStyleSmellDetector" class operates similarly to the "PMDSmellDetector" class but is tailored for the CheckStyle tool's specific output format. In the "findSmells" method, after loading the necessary library files, it builds and executes a command to run CheckStyle, capturing the output in an XML document, just like PMD (image 10).

```
public void findSmells(SmellType smellType, Map<SmellType, Set<Smell>> detectedSmells) throws URISyntaxException {
    File checkStyleJarFile = new File("checkstyle-10.12.1/checkstyle-10.12.1-all.jar");
    File checkStyleConfigFile = new File(getClass().getClassLoader().getResource("checkstyle-10.12.1/checkstyle-config.xml").toURI());


    String toolOutput = null;
    try {
        toolOutput = Utils.runCommand(buildToolCommand(checkStyleJarFile, checkStyleConfigFile), null, true);
    } catch (InterruptedException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    Document xmlDoc = Utils.getXmlDocument(toolOutput);

    try {
        extractSmells(smellType, xmlDoc, detectedSmells);
    } catch (Exception e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }
    this.detectedSmells = detectedSmells;
    System.out.println("End CheckStyle");

}
```

*Image 10 – The "findSmells" method of the "CheckStyleSmellDetector" class.*

The "extractSmells" method processes the XML document to detect and register the various types of code smells. It retrieves the <file> nodes from the XML and extracts information about the file, from which it generates a File object and extracts the class name. Next, it looks for <error> nodes within each file and based on the source of the error, the method determines the type of the smell (SmellType) and outputs the start line of the problem. For smells of type God Class, it simply registers the smell, while

for smells of type Long Parameter List, it retrieves the method name and creates smell objects, which it adds to the detectedSmells map (image 11).

```
private void extractSmells(SmellType smellType, Document xmlDoc, Map<SmellType, Set<Smell>> detectedSmells) throws Exception {
    NodeList fileNodes = xmlDoc.getDocumentElement().getElementsByTagName("file");
    for(int i = 0; i < fileNodes.getLength(); i++) {
        Node fileNode = fileNodes.item(i);

        String fileName = fileNode.getAttributes().getNamedItem("name").getNodeValue();
        String filePath = fileName.substring(fileName.indexOf("\\", fileName.indexOf(projectDirectory + 1)));
        File targetFile = new File(filePath);
        String className = fileName.substring(fileName.lastIndexOf('\\') + 1).replace(".java", "");

        NodeList errorNodes = fileNode.getChildNodes();
        for(int j = 0; j < errorNodes.getLength(); j++) {
            Node errorNode = errorNodes.item(j);
            //This is needed because there are unexpected children
            if(!errorNode.getNodeName().equals("error"))
                continue;

            String source = errorNode.getAttributes().getNamedItem("source").getNodeValue();
            SmellType detectedSmellType = MAP_FROM_DECTECTED_SMELLS_TO_SMELLTYPE.get(source.substring(source.lastIndexOf('.') + 1));

            int startLine = Integer.parseInt(errorNode.getAttributes().getNamedItem("line").getNodeValue());

            if(detectedSmellType == SmellType.GOD_CLASS) {
                //CheckStyle returns line 1 in case a GodClass is found, instead of the line in which the class is declared
                Utils.addSmell(detectedSmellType, detectedSmells, getDetectorName(),
                        Utils.createSmellObject(detectedSmellType, className, targetFile, startLine));
            } else {
                String methodName = "";
                if(detectedSmellType == SmellType.LONG_PARAMETER_LIST) {
                    methodName = (String) Utils.extractMethodNameAndCorrectLineFromFile(targetFile, startLine)[0];
                } else {
                    String message = errorNode.getAttributes().getNamedItem("message").getNodeValue().replace("Method ", "");
                    methodName = message.substring(0, message.indexOf(" "));
                }

                Utils.addSmell(detectedSmellType, detectedSmells, getDetectorName(),
                        Utils.createSmellObject(detectedSmellType, className, methodName, targetFile, startLine));
            }
        }
    }
}
```

*Image 11 – The "extractSmells" method of the "CheckStyleSmellDetector" class.*

The "DuDeSmellDetector" class detects duplicate code smells using the DuDe tool. In the "findSmells" method, it loads the necessary library files and builds and executes a command to run DuDe. It, then, searches for up to three result files (Result0.xml, Result1.xml, Result2.xml) in the DuDe directory. If a result file exists, it loads it as an XML document and calls the "extractDuplicates" method to process them.

```
public void findSmells(SmellType smellType,  Map<SmellType, Set<Smell>> detectedSmells) throws Exception {
    File dudeJarFile = new File("dude/dude.jar");
    File dudeConfigFile = new File(getClass().getClassLoader().getResource("dude/selected-project.txt").toURI());

    writeSelectedProjectPathToConfigFile(dudeConfigFile);

    String dudeDirectory = dudeJarFile.getAbsolutePath().toString();
    dudeDirectory = dudeDirectory.substring(0, dudeDirectory.lastIndexOf("\\") + 1);

    Utils.runCommand(buildToolCommand(dudeJarFile, dudeConfigFile), dudeDirectory, false);

    for(int i = 0; i < 3; i++) {
        File resultsFile = new File(String.format(dudeDirectory + "Result%d.xml", i));
        if(!resultsFile.exists())
            continue;

        Document xmlDoc = Utils.getXmlDocument(resultsFile);
        extractDuplicates(xmlDoc, detectedSmells);
        resultsFile.delete();
    }
    this.detectedSmells = detectedSmells;
    System.out.println("End DuDe");
}
```

*Image 12 – The "findSmells" method of the "DuDeSmellDetector" class.*

The "extractDuplicates" method processes the document to detect duplicate pieces of code and adds them to the detectedSmells map. It parses <DupChain> and

<CodeSnippet> nodes, focusing only on .java files, extracts details such as the class name, the start and end lines of the duplicate, and creates sniff objects of type DUPLICATE_CODE and adds them to the map.

```
private void extractDuplicates(Document xmlDoc, Map<SmellType, Set<Smell>> detectedSmells) throws Exception {
    int duplicationGroupId = Utils.getGreatestDuplicationGroupId(detectedSmells);

    NodeList dupChains = xmlDoc.getDocumentElement().getElementsByTagName("DupChain");
    dupChainLoop: for(int i = 0; i < dupChains.getLength(); i++) {
        NodeList codeSnippets = dupChains.item(i).getChildNodes();
        for(int j = 0; j < codeSnippets.getLength(); j++) {
            Node codeSnippet = codeSnippets.item(j);

            if(!codeSnippet.getNodeName().equals("CodeSnippet"))
                continue;

            String fileName = codeSnippet.getAttributes().getNamedItem("FileName").getNodeValue();
            if(!fileName.endsWith(".java"))
                continue dupChainLoop;

            String className = fileName.substring(fileName.lastIndexOf("\\") + 1);
            File targetIFile = new File(fileName);
            int startLine = Integer.parseInt(codeSnippet.getAttributes().getNamedItem("From").getNodeValue());
            int endLine = Integer.parseInt(codeSnippet.getAttributes().getNamedItem("To").getNodeValue());

            Utils.addSmell(SmellType.DUPLICATE_CODE, detectedSmells, getDetectorName(),
                    Utils.createSmellObject(SmellType.DUPLICATE_CODE, duplicationGroupId, className, targetIFile, startLine, endLine));
        }

        duplicationGroupId++;
    }
}
```

*Image 13 – The "extractDuplicates" method of the "DuDeSmellDetector".*

The "Utils" class provides essential utility functions for code smell detection, including running system commands, parsing XML data, creating and managing Smell objects, and handling specific code analysis tasks like determining line numbers and method names. It centralizes common operations needed across different parts of the detection system, facilitating modularity and code reuse. The detectors call for its methods in various points, but especially in their "extractSmells" methods where they create a new smell and add it to the map.

```
public static Smell createSmellObject(SmellType smellType, Object... args) throws Exception {
    Smell.Builder codeSmellBuilder;

    /** @formatter: off */
    if(smellType == SmellType.DUPLICATE_CODE) {
        codeSmellBuilder = new Smell.Builder(smellType)
                                    .setDuplicationGroupId((Integer) args[0])
                                    .setClassName((String) args[1])
                                    .setTargetFile((File) args[2])
                                    .setStartLine((Integer) args[3])
                                    .setEndLine((Integer) args[4]);
    } else if(Utils.isClassSmell(smellType)) {
        codeSmellBuilder = new Smell.Builder(smellType)
                                    .setClassName((String) args[0])
                                    .setTargetFile((File) args[1])
                                    .setStartLine((Integer) args[2]);
    } else {
        codeSmellBuilder = new Smell.Builder(smellType)
                                    .setClassName((String) args[0])
                                    .setMethodName((String) args[1])
                                    .setTargetFile((File) args[2])
                                    .setStartLine((Integer) args[3]);
    }
    /** @formatter: on */

    return codeSmellBuilder.build();
}
```

*Image 14 – The "createSmellObject" method of the "Utils" class.*

The "createSmellObject" method creates a Smell object based on the type of smell and the provided parameters. Depending on the type of the smell (SmellType), it uses

different parameters to build the object. For example, for duplicate code types it uses parameters such as the duplicate group ID, class name, file, and start and end lines, while for class-related scents, it uses parameters such as the class name, file, and start line. For other types of scents associated with methods, it uses the class name, method name, file, and start line. The “addSmells” method adds the new smell in the designated map, after checking though for the possibility of duplicates.

```
public static void addSmell(SmellType smellType, Map<SmellType, Set<Smell>> detectedSmells, String detectorName, Smell newSmell) {
    if(!detectedSmells.containsKey(smellType)) {
        detectedSmells.put(smellType, new LinkedHashSet<Smell>());
    }

    Set<Smell> detectedSmellsForSmellType = detectedSmells.get(smellType);
    if(detectedSmellsForSmellType.contains(newSmell)) {
        detectedSmellsForSmellType.stream().filter( smell -> smell.equals(newSmell)).findFirst().orElse(null).addDetectorName(detectorName);
    } else {
        newSmell.addDetectorName(detectorName);
        detectedSmellsForSmellType.add(newSmell);
    }
}
```

*Image 15 – The “addSmell” method of the “Utils” class.*

One of the most important methods in the whole project is the “runCommand” method. Three detectors rely on this method for their proper operation (PMD, CheckStyle, DuDe). The method executes a command specified by the commandList in a specified directory, using ProcessBuilder. If returnOutput is true, the method reads the output of the command, ignoring specific lines associated with the CheckStyle tool, and returns the remaining result as a string. If returnOutput is false, the method waits up to two minutes for the command to complete and then terminates the process. If the process does not terminate correctly, it resorts to a forced termination. If any error occurs during execution, it prints the error.

```
public static String runCommand(List<String> commandList, String directory, boolean returnOutput) throws InterruptedException {
    ProcessBuilder pb = new ProcessBuilder(commandList);
    pb.redirectErrorStream(true);
    if(directory != null && !directory.isEmpty())
        pb.directory(new File(directory));

    StringBuilder output = new StringBuilder();
    try {
        Process p = pb.start();

        if(!returnOutput) {
            p.waitFor(2, TimeUnit.MINUTES);
            p.destroy();
            if(!p.waitFor(20, TimeUnit.SECONDS))
                p.destroyForcibly();

            return null;
        }

        BufferedReader reader = new BufferedReader(new InputStreamReader(p.getInputStream()));

        String line;
        while((line = reader.readLine()) != null) {
            //Skipping unnecessary line from CheckStyle tool
            if(line.startsWith("Checkstyle ends with"))
                continue;

            output.append(line);
            output.append("\n");
        }

        reader.close();
        p.destroy();
    } catch (IOException e) {
        e.printStackTrace();
    }

    return output.toString();
}
```

*Image 16 – The “runCommand” method of the “Utils” class.*

After the detection process is completed the “Main” class, as stated above, creates an instance of the “SmellResultsFrame” class and the “SmellExporter” class.

The “SmellResultsFrame” class is responsible for showing the detected smells. In order to do that it creates a new frame “Detected Smells”. In the frame a JTable is created to show the detected smells. The table consists of the following columns: "Smell Type", "Class Name", "Affected Element", "Start Line", and "Detectors". For the better handling of the results two filters were added, one that filters the table according to the number of the smell detectors that found a smell, and one that filters the table according to a specific detector. For both filters a JComboBox is used to show the user his choices.

```
filterComboBox = new JComboBox<>(new Integer[]{0, 2, 3, 4}); // 0 means no filter
filterComboBox.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        int selectedNumber = (int) filterComboBox.getSelectedItem();
        String selectedDetector = (String) detectorComboBox.getSelectedItem();
        populateTable(selectedNumber, selectedDetector);
    }
});
```

*Image 17 – The filter according to the number of the detectors.*

```
Set<String> detectors = detectedSmells.values().stream()
        .flatMap(Set::stream)
        .flatMap(smell -> Set.of(smell.getDetectorNames().split(",")).stream())
        .collect(Collectors.toCollection(TreeSet::new));
detectors.add("All Detectors"); // Option to not filter by detector
detectorComboBox = new JComboBox<>(detectors.toArray(new String[0]));
detectorComboBox.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        int selectedNumber = (int) filterComboBox.getSelectedItem();
        String selectedDetector = (String) detectorComboBox.getSelectedItem();
        populateTable(selectedNumber, selectedDetector);
    }
});
```

*Image 18 – The filter according to a specific detector.*

The class uses the “populateTable” method to populate the table with rows filtered by the number of detectors and specific detector name.

```
private void populateTable(int detectorCount, String detectorName) {
    model.setRowCount(0); // Clear existing rows

    for (Map.Entry<SmellType, Set<Smell>> entry : detectedSmells.entrySet()) {
        SmellType smellType = entry.getKey();
        Set<Smell> smells = entry.getValue();

        for (Smell smell : smells) {
            // Check the number of detectors if a specific filter is applied
            int numDetectors = smell.getDetectorNames().split(",").length;
            boolean matchesDetectorCount = detectorCount == 0 || numDetectors == detectorCount;

            // Check if the smell was detected by the selected detector
            boolean matchesDetectorName = detectorName.equals("All Detectors") || smell.getDetectorNames().contains(detectorName);

            if (matchesDetectorCount && matchesDetectorName) {
                Object[] rowData = {
                    smellType.getName(),
                    smell.getClassName(),
                    smell.getAffectedElementName(),
                    smell.getTargetStartLine(),
                    smell.getDetectorNames()
                };
                model.addRow(rowData);
            }
        }
    }
}
```

*Image 19 – The “populateTable” method of the “SmellResultsFrame” class.*

Lastly the “SmellExporter” class is designed to export detected code smells into CSV files for reporting. It aggregates the detected smells by affected class and file path, and then writes two CSV files: one listing aggregated data with smell types and detectors (image 20), and another with separate columns for each type of detector (image 21). The class processes each detected smell, keeps track of which detectors found each smell, and counts occurrences for each detector. These two CSV files are the files that will be analyzed and examined in the next section.

```
private static void writeCSV(String filePath, Map<String, AggregatedSmells> aggregatedData) {
    try (FileWriter writer = new FileWriter(filePath)) {
        // Write the header
        writer.append("class_name")
              .append(';')
              .append("class_path")
              .append(';')
              .append("SmellType")
              .append(';')
              .append("DetectorNames")
              .append(';')
              .append("Count")
              .append('\n');

        // Write the data rows
        for (AggregatedSmells aggregatedSmells : aggregatedData.values()) {
            writer.append(aggregatedSmells.getAffectedElement())
                  .append(';')
                  .append(aggregatedSmells.getClassPath())
                  .append(';')
                  .append(String.join(" | ", aggregatedSmells.getSmellTypes()))  // Updated to get all smell types
                  .append(';')
                  .append(aggregatedSmells.getDetectorNames())
                  .append(';')
                  .append(String.valueOf(aggregatedSmells.getCount()))
                  .append('\n');
        }
    } catch (IOException e) {
        System.err.println("Error writing CSV file: " + e.getMessage());
    }
}
```

*Image 20 – The method responsible for writing the first CSV file.*

```
private static void writeDetectorSpecificCSV(String filePath, Map<String, AggregatedSmells> aggregatedData) {
    try (FileWriter writer = new FileWriter(filePath)) {
        // Write the header
        writer.append("class_name")
              .append(';')
              .append("class_path")
              .append(';')
              .append("SmellType")
              .append(';')
              .append("DetectorOrganic")
              .append(';')
              .append("DetectorPMD")
              .append(';')
              .append("DetectorCheckStyle")
              .append(';')
              .append("DetectorDuDe")
              .append('\n');

        // Write the data rows
        for (AggregatedSmells aggregatedSmells : aggregatedData.values()) {
            writer.append(aggregatedSmells.getAffectedElement())
                  .append(';')
                  .append(aggregatedSmells.getClassPath())
                  .append(';')
                  .append(String.join(" | ", aggregatedSmells.getSmellTypes()))  // Updated to get all smell types
                  .append(';')
                  .append(String.valueOf(aggregatedSmells.getDetectorOrganicCount()))
                  .append(';')
                  .append(String.valueOf(aggregatedSmells.getDetectorPMDCount()))
                  .append(';')
                  .append(String.valueOf(aggregatedSmells.getDetectorCheckStyleCount()))
                  .append(';')
                  .append(String.valueOf(aggregatedSmells.getDetectorDuDeCount()))
                  .append('\n');
        }
    } catch (IOException e) {
        System.err.println("Error writing CSV file: " + e.getMessage());
    }
}
```

*Image 21 – The method responsible for writing the second CSV file.*

## 4.3. User Interface

In order to use the project, one must have the Eclipse IDE downloaded, along of course with the Java programming language. After the downloading of the project, you must import it in the eclipse as an “Existing Projects into Workspace” (image 22).

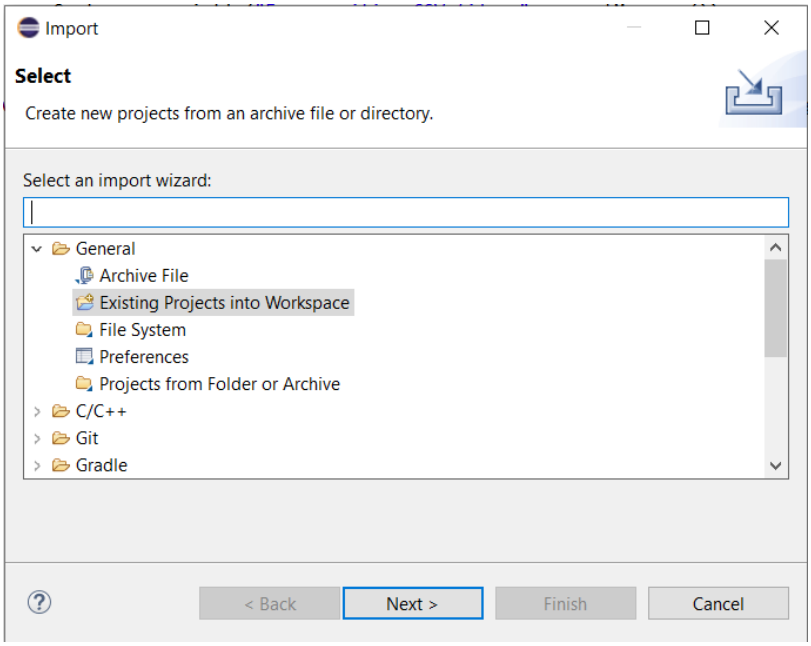


*Image 22 – Importing the project*

After the import, you must add some needed jar files in order for the project to work. These files can be found in the plugins folder. In order to open the “Properties for SmellDetector” window, right-click on the project, click on “Build Path” and then on “Configure Build Path…”. There on the “Libraries” tab, by clicking first on the “Classpath” and then on the “Add External JARs” you can add the needed jar files.

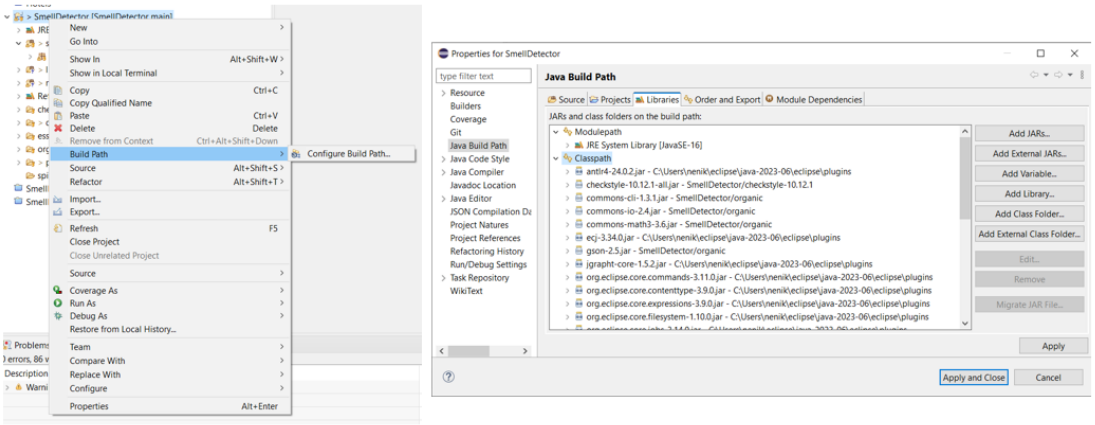


*Image 23 – Adding the necessary JAR files.*

Once this process is over you can run the project. A window will open, so that you can add the path to the project you wish to analyze, either by typing on the field or by browsing. After filling in the fields you can click on the “Select Smell Detector” button

to select the smell detector tool. It should be noted that it will be better if the paths do not include characters from other languages, only English. A new window will be opened where you can choose a smell detector. There are 5 options, All Smells, PMD, Organic, DuDe and Checkstyle.

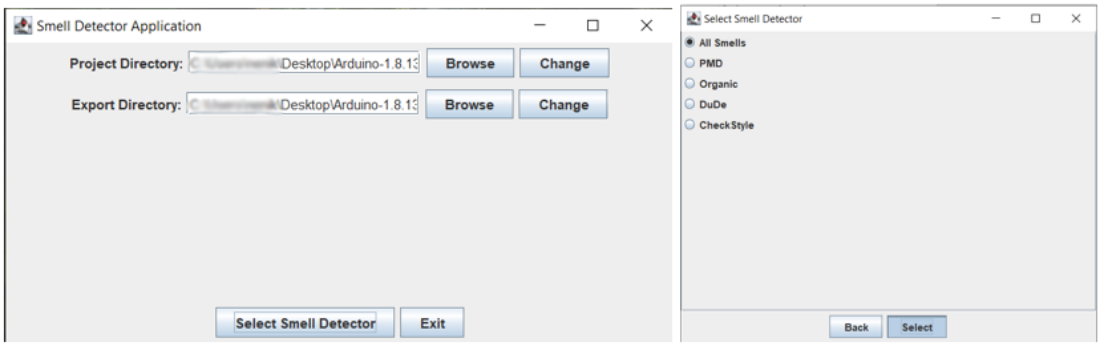


*Image 24 – Using the Smell Detector Application.*

After the completion of the detection process a pop-up message will notify you. Once you click the OK button, the project automatically exports the results in CSV files in the selected export directory. At the same time the app shows the results in a table in a new window.

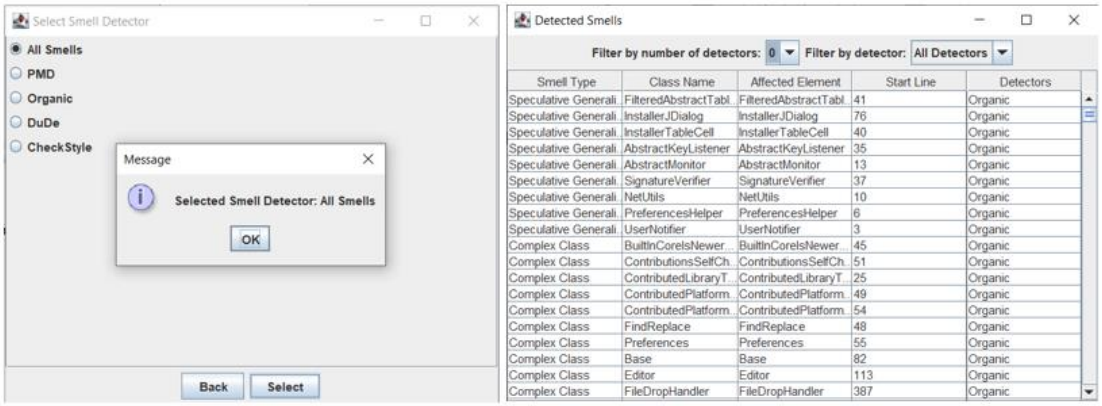


*Image 25 – Showing the results after the detection is completed.*

In the new window “Detected Smells”, there are two filters for the smells. The first one filters by number of detectors that found one smell and it has 4 choices, 0, 2, 3 and 4. “0” means no filter, 2 that 2 detectors found a smell, 3 that 3 detectors found a smell and 4 that all the detectors found the smell. The second filter keeps the smells that were detected by a specific detector.

# 5. Case Study

This section describes how the data were extracted and processed. It, also, provides details on how the correlations between the data were extracted and finally it analyses the conclusions drawn from them.

## 5.1. Data Extraction and Processing

The data that were extracted from the 25 projects (table 7) using the smell detector tools were stored in two csv files. The first file had information about the classes on the smell types that were detected, the detectors that found those smells and a counter that stored how many times any kind of smell was detected from any detector. The second file had information about the classes on the smell types that were detected and four counters, one for each detector, that stored how many times any kind of smell was detected from one specific detector. Both files before processing had a total of 25.239 lines of data.

| Project Name | Description | LoC |
|---|---|---|
| arduino | Physical computing platform | 27K |
| arthas | Java Diagnostic tool | 28K |
| azkaban | Workflow manager | 79K |
| cayenne | Java object to relational mapping framework | 348K |
| deltaspike | CDI management | 146K |
| exoplayer | Android media player | 155K |
| gson | Java library to convert Java Objects to JSON | 25K |
| javacv | Wrappers of commonly used libraries | 23K |
| jclouds | Toolkit for java cloud applications | 482K |
| joda-time | Date and time handling | 86K |
| libgdx | Game development framework | 280K |
| maven | Software project management tool | 106K |
| mina | Network application framework | 35K |
| nacos | Cloud application microservices build and management | 60K |
| opennlp | Natural Language Processing toolkit | 93K |
| openrefine | Data management | 69K |
| pdfbox | Library of processing pdf documents | 213K |
| redisson | Java Redis client and Netty framework | 133K |
| RxJava | Composing asynchronous and event-based programs with observable sequences | 310K |
| testng | Testing framework | 85K |
| vassonic | Performance framework for mobile websites | 7K |
| wss4j | Java implementation for security standards in web applications | 136K |
| xxl-job | Distributed task scheduling framework | 9K |
| zaproxy | Security tool | 187K |

*Table 7 – The 25 projects used.*

When the data were extracted, it was obvious that there were duplicated lines that differ in the SmellType and the DetectorNames categories. Therefore, the processing step involved removing these duplicates and merging them into one line. After this, the two files had a total of 20.782 lines of data. Then the two datasets were merged with the dataset from the paper of Tsoukalas et al. This dataset contained various metrics on the classes of the 25 projects.

After merging the two csv files with the dataset created by Tsoukalas et al., there were two new csv files, that contained both the metrics from the table and the results from the smell detector tools. In the end the merged datasets had 9.909 lines of data each. Although, the number decreased significantly, the dataset with the results from the detectors had information not only on the classes of the projects, but also from methods in the classes and therefore the final number of lines is smaller.

To extract the correlations, the python language was used. More specifically the function “correlations” was used, that summarizes the strength and direction of the linear (straight-line) association between two quantitative variables.

## 5.2. Agreement between the Smell Detector tools

The agreement between the smell detector tools was checked before the two datasets were merged and the results from the check are shown in table 8. The highest correlation is between the detectors PMD and DuDe (0,637021). Their high correlation suggests that they have a relatively strong positive relationship with each other. This makes sense as they are the only ones that detect duplicate code smells. At this point it must be highlighted that the detector DuDe did not have any common smell types with the detectors Organic and CheckStyle, something that explains its low correlation with them. Apart from this strong relationship, however, all other relationships are weak to almost zero.

For example, the Organic detector does not have any strong relationships with the other detectors. This can be explained by the fact that the detector can detect a total of 17 types of smells and only 3 of them are in common with the other detectors. Its strongest correlation is with the PMD detector, but it is relatively low (0,274151). As for the CheckStyle detector the strongest relationship it has is, with the Organic detector, but it is very weak (0,062595). What is interesting here is the fact that even though CheckStyle has all its types of smells in common with the PMD their relationship is almost zero (0,013649). This might be because of how each of them detects the smells.

| | **Detector Organic** | **Detector PMD** | **Detector CheckStyle** | **Detector DuDe** |
|---|---|---|---|---|
| **Detector Organic** | 1,000000 | 0,274151 | 0,062595 | 0,290626 |
| **Detector PMD** | 0,274151 | 1,000000 | 0,013649 | 0,637021 |
| **Detector CheckStyle** | 0,062595 | 0,013649 | 1,000000 | 0,007824 |
| **Detector DuDe** | 0,290626 | 0,637021 | 0,007824 | 1,000000 |

*Table 8 – Correlations between Smell Detectors*

Below are graphs that show the relationships between the smell detectors.

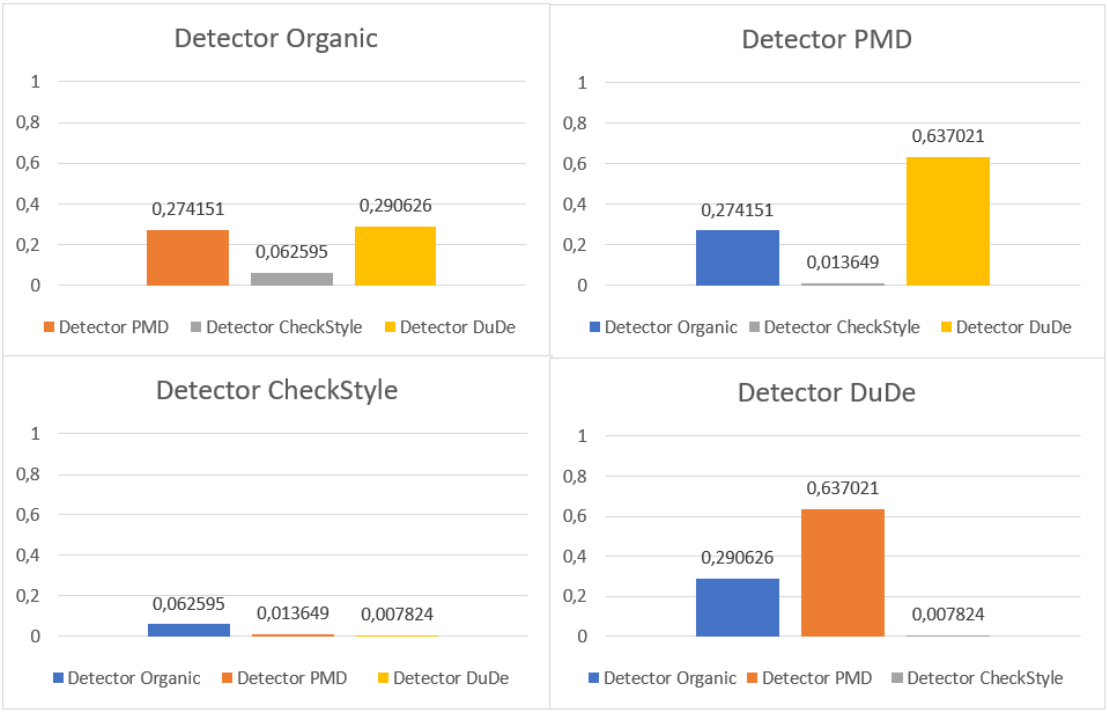


*Image 26 – Graphs showing the correlations between the detectors.*

This work, also, checked for correlations between the counters of the detectors with the metrics. Basis was given to the correlations with the metric Max Ruler, which was expected to be higher. Table 9 shows these correlations in detail.

| Metrics | Counter |
|---|---|
| commits_count | 0,341207 |
| code_churn_avg | 0,357163 |
| contributors_count | 0,582116 |
| contributors_experience | -0,065347 |
| hunks_count | 0,138841 |
| issue_tracker_issues | 0,055174 |
| cbo | 0,473491 |
| wmc | 0,511957 |
| dit | 0,056791 |
| rfc | 0,586868 |
| lcom | 0,108008 |
| max_nested_blocks | 0,375198 |
| total_methods | 0,297313 |
| total_variables | 0,54223 |
| total_refactorings | 0,288365 |
| duplicated_lines_density | 0,43407 |
| comment_lines_density | 0,387856 |
| ncloc | 0,698095 |
| Max_Ruler | 0,451934 |

*Table 9 – Correlations between the metrics of the classes and the counter of all detectors.*

The results from table 9 can be categorized into five main groups in relation with code smells: negative correlations, very low or negligible correlations, low positive correlations, moderate positive correlations, strong positive correlations. The first category includes only the metric “contributors_experience” with a correlation of -0,065347. This suggests that the experience of the contributors has no influence on the quality of the code. The second category, very low or negligible correlations with code smells, includes the metrics “lcom” (0,108008), dit (0,056791) and issue_tracker_issues (0,055174). The metrics in this category do not seem to be directly related with the quality of the code. The third category, low positive correlations with code smells, includes the metrics total_methods (0,297313), total_refactorings (0,288365), commits_count (0,341207) and hunks_count (0,138841). Here, the correlations suggest only a slight relationship between the metrics and the presence of code smells.

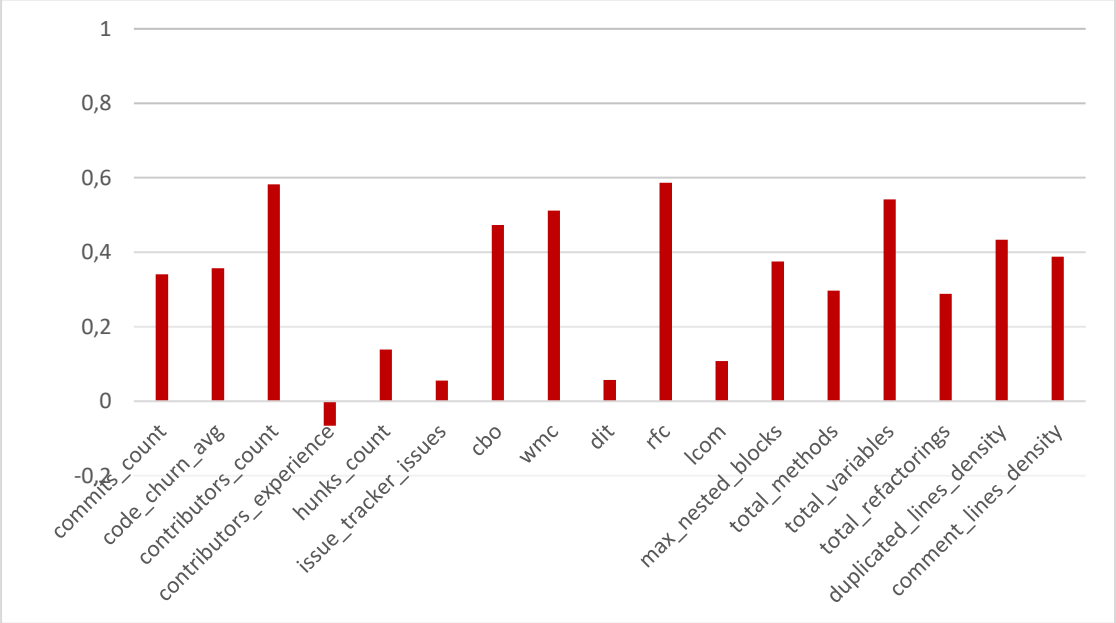


*Image 27 – Graph showing the correlations between the detectors and the software metrics.*

In the next category, moderate positive correlations with code smells, there seems to be a relationship, but not strong enough. Max Ruler (0,451934) belongs in this category, along with cbo (0,473491), duplicated_lines_density (0,434070), comment_lines_density (0,387856), max_nested_blocks (0,375198) and code_churn_avg (0,357163). The last category, strong positive correlations with code smells, includes the metrics ncloc (0,698095), rfc (0,586868), contributors_count (0,582116), total_variables (0,542230) and wmc (0,511957). The metrics, that belong here, have a strong positive correlation with the number of code smells, suggesting that as these metrics increase, the number of code smells in a class is also likely to increase. It is easy to conclude that classes that have one or more of the following characteristics are more likely to have more smells: large amounts of lines of code, many method’s interactions, a higher number of variables and a higher number of methods. These metrics indicate that the code is large and complex and therefore of substandard quality. It is, also, shown that the number of contributors is positively correlated with code

smells, implying that classes worked on by more developers tend to have more smells, potentially due to varying coding practices.

Next, an investigation was made on the agreement between the metrics with each of the smell detectors separately. Table 10 shows the correlations between the metrics of the classes with each counter of the detectors. More specifically, it seems that there is a strong relationship between the metrics “rfc” (0,685571), “total_variables” (0,60111) and “ncloc” (0,644641), and the Organic Smell Detector. There is, also, a strong correlation between the metrics “lcom” (0,823732) and “total_methods” (0,858807), and the CheckStyle Smell Detector. On the contrary, the detectors PMD and DuDe do not seem to have any strong correlation with the metrics, except only the correlation between “duplicated_lines_density” (0,512053) and the DuDe Smell Detector, something that’s expected. The DuDe detector has also a moderate correlation with the metric “code_churn_avg”. The “Max_Ruler” metric has a moderate correlation with the detectors.

| Metrics | Organic Counter | PMD Counter | CheckStyle Counter | DuDe Counter |
|---|---|---|---|---|
| commits_count | 0,453053 | 0,155437 | 0,127799 | 0,058774 |
| code_churn_avg | 0,061123 | 0,076614 | 0,063587 | 0,536798 |
| contributors_count | 0,339474 | 0,10712 | 0,137183 | 0,004621 |
| contributors_experience | -0,138992 | 0,027854 | -0,063173 | 0,02931 |
| hunks_count | 0,162483 | 0,102661 | 0,062294 | 0,028708 |
| issue_tracker_issues | 0,07174 | 0,023592 | 0,033862 | 0,008545 |
| cbo | 0,498865 | 0,270795 | 0,273271 | 0,17128 |
| wmc | 0,471098 | 0,248366 | 0,68104 | 0,181517 |
| dit | 0,036149 | 0,132128 | 0,016589 | 0,010099 |
| rfc | 0,685571 | 0,298776 | 0,137352 | 0,204996 |
| lcom | -0,004188 | 0,002062 | 0,823732 | 0,00356 |
| max_nested_blocks | 0,456437 | 0,183893 | 0,07565 | 0,11803 |
| total_methods | 0,163664 | 0,125557 | 0,858807 | 0,11621 |
| total_variables | 0,60111 | 0,296569 | 0,154572 | 0,208165 |
| total_refactorings | 0,350895 | 0,107875 | 0,128406 | 0,086895 |
| duplicated_lines_density | 0,097276 | 0,419337 | 0,03556 | 0,512053 |
| comment_lines_density | 0,3023 | 0,175989 | 0,105836 | 0,295367 |
| ncloc | 0,644641 | 0,372627 | 0,343953 | 0,371098 |
| Max_Ruler | 0,45258 | 0,333659 | 0,156203 | 0,183213 |

*Table 10 - Correlations between the metrics of the classes and the counters of each detector.*

Generally, Organic seems to have a moderate correlation with the metrics. The metrics “rfc” (0,685571), “total_variables” (0,60111) and “ncloc” (0,644641) are the ones that seem to have the biggest correlation with the detector. Organic is a detector that focuses on the complexity of the code and as explained on section 2 so are these metrics. Max Ruler (0,45258) has the higher correlation with the Organic tool that with all the

others. However, their correlation is still moderate, perhaps because although metrics focus on code quality and maintenance, they evaluate different aspects and use different methods to measure complexity and technical debt.

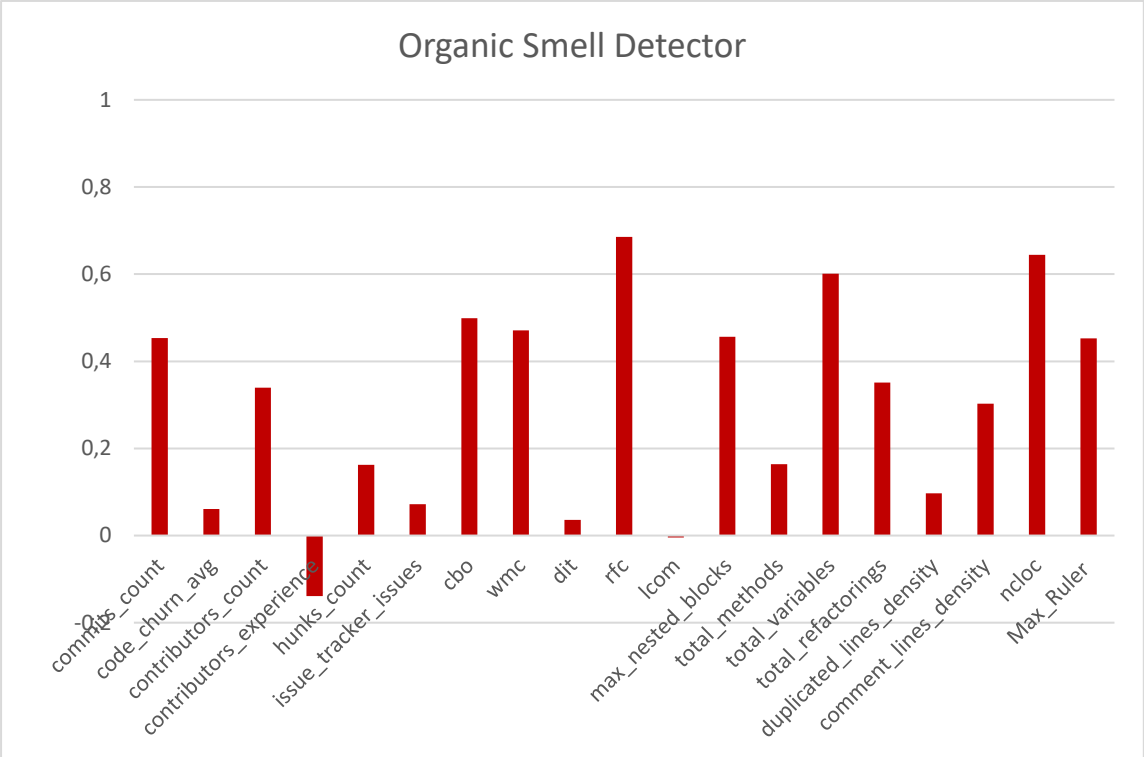


*Image 28 – Graph showing the correlations between the Organic Detector and the software metrics.*

The results as regards the PMD detector are rather disappointing. The highest correlation is with the "duplicated_lines_density" metric (0,419337), but it is not strong enough to extract reliable conclusions. Overall, the results show a variation in correlation, indicating that PMD only detects specific aspects of code complexity and quality, that differ from those of the metrics.

CheckStyle has too many variations in its correlations, with values ranging from -0,063 to 0,858. Its strong correlations with the "lcom" and "total_methods" metrics indicate that the detector can identify certain smells, associated with methods, with a high level of effectiveness. On the other hand, it seems that the detector does not have the ability to detect the majority of the remaining problematic code fragments found by the metrics. In summary, the results show that CheckStyle has a strong effect on some aspects of the code, while its ability to detect other aspects is more limited or uncertain.

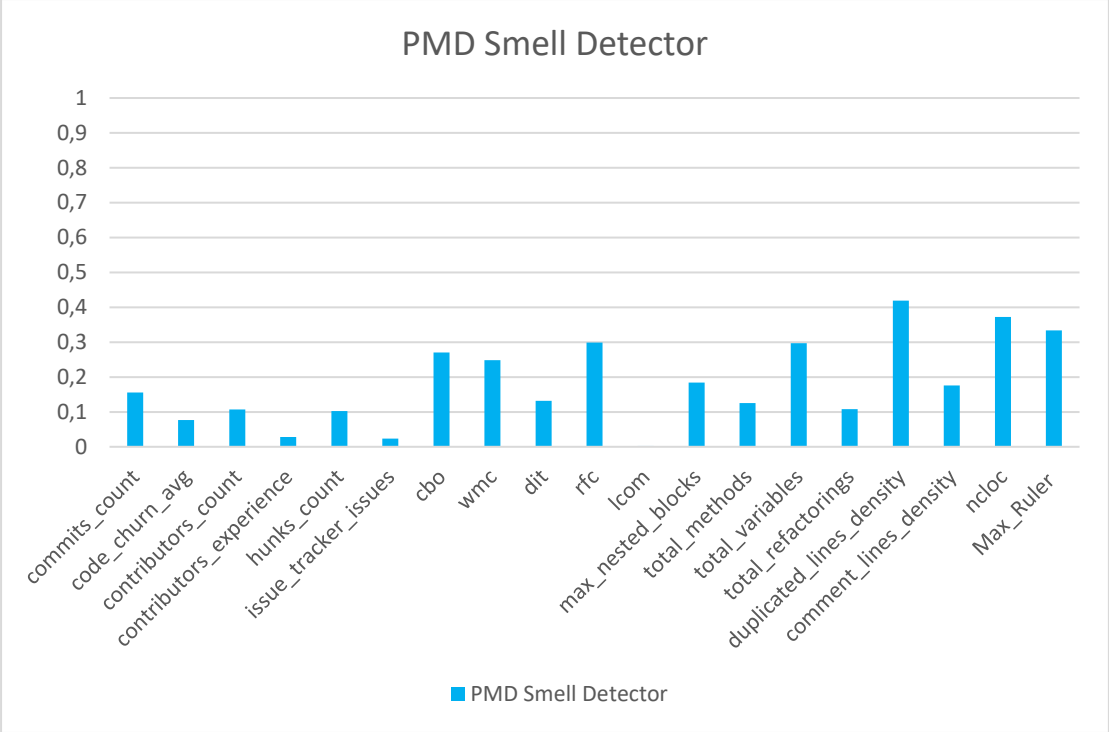


*Image 29 – Graph showing the correlations between the PMD Detector and the software metrics.*

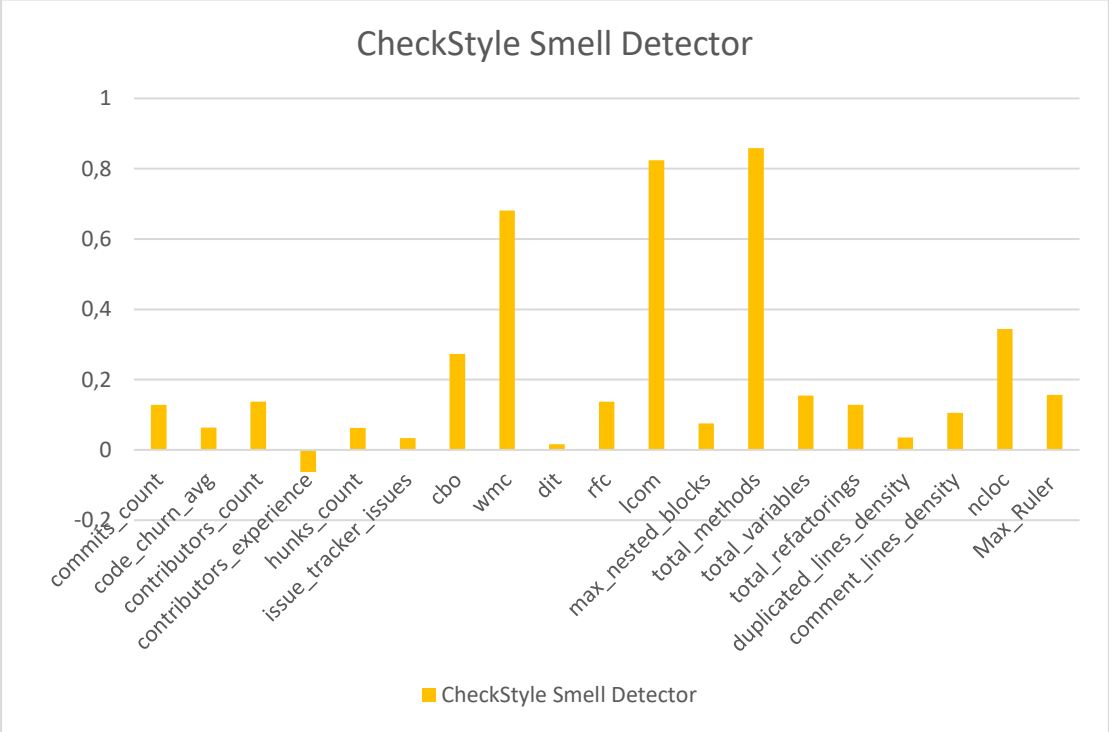


*Image 30 – Graph showing the correlations between the CheckStyle Detector and the software metrics.*

Last but not least, DuDe does not have strong correlations with the metrics. This is expected, though, as it only detects “duplicate code” type of smell. The tool has the strongest correlations with the “code_churn_avg” and “duplicated_lines_density” metrics, that even though are relatively moderate they can be explained due to the unique way the detector identifies duplicated code. Overall, it makes sense that it has its strongest agreement with the metrics that count the size of a code churn and the percentage of lines in a file involved in duplications. The larger the size of a code churn, the greater the chance of duplicate code.

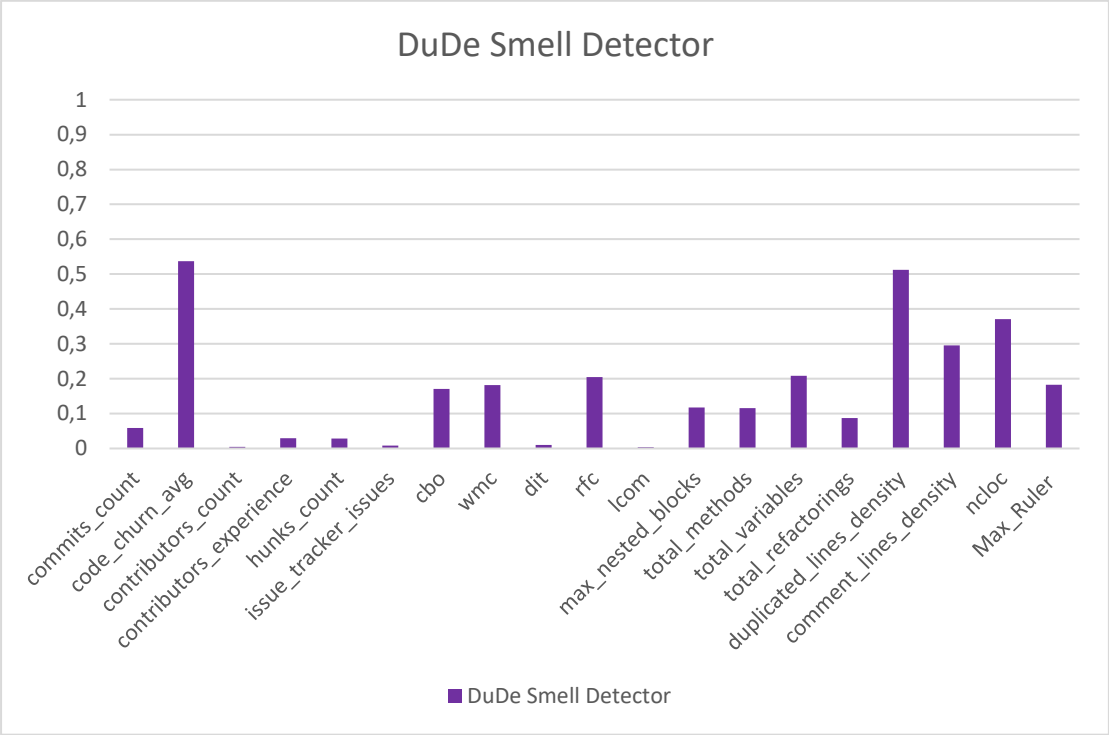


*Image 31 – Graph showing the correlations between the DuDe Detector and the software metrics.*

# 6. Conclusion

## 6.1. Summary and conclusions

Smell detection is an important part of the TD assessment process but despite the large number of detection tools it is still complex and time-consuming. One solution to this is the combination of detectors. In this work 4 detectors, Organic, PMD, CheckStyle and DuDe, were combined in a java project and their agreement with each other and with software metrics was investigated. The results were relatively modest but interesting at the same time. The detectors with the highest agreement were PMD and DuDe, which is attributed to their common Duplicate Code smell. Organic was relatively independent, while no reliable conclusions can be drawn from CheckStyle, as the correlations with the other detectors were almost zero.

A similar situation prevails in metrics research. The conclusions that can be drawn here are that classes that have one or more of the following characteristics are more likely to have more smells: large amounts of lines of code, many method's interactions, a higher number of variables and a higher number of methods.

## 6.2. Limitations of the research

As mentioned before, this work is a continuance of the study by Ichtsis et al., but it has some limitations. The project developed does not use the smell detection tools JDeodorant and JSpIRIT, as their structure as plugins restricts them from being used in a simple java project. Thus, the results are missing the associations with these two tools.

## 6.3. Future Work

This research can form the basis for the creation of a machine learning model that will find problematic points in the code based on software metrics. Future work could involve not only developing and training such a model using a comprehensive dataset of labeled code examples but also integrating various software metrics to enhance accuracy.

# 7. About Bibliography

## 7.1. References


1. (Cunningham, 1992) Cunningham, W. (1992). The WyCash portfolio management system. Addendum to the proceedings on Object-oriented programming systems, languages, and applications (Addendum) - OOPSLA '92. doi: https://doi.org/10.1145/157709.157715 .
2. Arcelli Fontana, F., Braione, P. and Zanoni, M. (2012). Automatic detection of bad smells in code: An experimental assessment. The Journal of Object Technology, 11(2), p.5:1. doi: https://doi.org/10.5381/jot.2012.11.2.a5 .
3. Rasool, G. and Arshad, Z. (2015). A review of code smell mining techniques. Journal of Software: Evolution and Process, [online] 27(11), pp.867–895. doi: https://doi.org/10.1002/smr.1737 .
4. Hamid, A., Ilyas, M., Hummayun, M. and Nawaz, A. (2013). A Comparative Study on Code Smell Detection Tools. International Journal of Advanced Science and Technology, 60, pp.25–32. doi: https://doi.org/10.14257/ijast.2013.60.03 .
5. Fernandes, E., Oliveira, J., Vale, G., Paiva, T. and Figueiredo, E. (2016). A review-based comparative study of bad smell detection tools. Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering. doi: https://doi.org/10.1145/2915970.2915984 .
6. Apostolos Ichtsis, Nikolaos Mittas, Apostolos Ampatzoglou and Chatzigeorgiou, A. (2022). Merging smell detectors. doi: https://doi.org/10.1145/3524843.3528089 .
7. Tsoukalas, D., Mittas, N., Chatzigeorgiou, A., Kehagias, D.D., Ampatzoglou, A., Amanatidis, T. and Angelis, L. (2021). Machine Learning for Technical

Debt Identification. IEEE Transactions on Software Engineering, pp.1–1. doi: https://doi.org/10.1109/tse.2021.3129355 .

8. docs.pmd-code.org. (n.d.). Documentation Index | PMD Source Code Analyzer. [online] Available at: https://docs.pmd-code.org/latest/ [Accessed 20 Aug. 2024].
9. Github.io. (2024). Opus Research Group: Tools. [online] Available at: https://opus-research.github.io/tools.html [Accessed 20 Aug. 2024].
10. checkstyle.sourceforge.io. (n.d.). checkstyle – Checkstyle 10.12.3. [online] Available at: https://checkstyle.sourceforge.io/index.html [Accessed 4 Sep. 2024].
11. R. Wettel and Marinescu, R. (2005). Archeology of code duplication: recovering duplication chains from small duplication fragments. [online] doi: https://doi.org/10.1109/synasc .2005.20.
12. Eclipse Plugins, Bundles and Products - Eclipse Marketplace. (n.d.). JDeodorant. [online] Available at: https://marketplace.eclipse.org/content/jdeodorant [Accessed 20 Aug. 2024].
13. Vazquez, H.C. (2020). hcvazquez/JSpIRIT. [online] GitHub. Available at: https://github.com/hcvazquez/JSpIRIT [Accessed 4 Sep. 2024].

## 7.2. Penalties for plagiarism

According to Thesis Regulation:

*"Plagiarism and misappropriation of foreign achievements are prohibited and we (the university) must act in a manner consistent with the requirements of the applicable legislation for the protection of intellectual property and inventions patented inventions (Act No. 1733/1987, 1883/1990, 2029/1992, 2128/1993, Decree 77/1988, 16/1991, 321/2001, Law 8121/1993)."*